

Федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 «Программная инженерия» –

Системное и прикладное программное обеспечение

Дисциплина: Рефакторинг баз данных и приложений

Этап 2: Модернизация кодовой базы

Выполнили:

студенты 4 курса

Кравцов Кирилл Денисович

Группа: Р3411

Батманов Даниил Евгеньевич

Группа: Р3407

Преподаватель:

Логинов Иван Павлович

г. Санкт-Петербург, 2025

2.1. Переписывание кода с Java на Kotlin

[Pull request](#)

Задача

Проблема: Java-код с большим количеством boilerplate, отсутствием null-safety и менее выразительным синтаксисом.

Цель:

- Получить более лаконичный и выразительный код
- Использовать null-safety для устранения NullPointerException на уровне компилятора
- Применить data classes для автоматической генерации equals, hashCode, toString, copy
- Сократить количество boilerplate кода
- Использовать современные возможности языка

Реализация: Полностью переписать весь код с Java на Kotlin, применяя идиоматичный стиль (data classes, properties, extension functions, scope functions).

Результаты

В рамках рефакторинга выполнен полный перенос кодовой базы с Java на Kotlin. Это значительное преобразование включало конвертацию всех исходных файлов с сохранением функциональности приложения и бизнес-логики. При миграции использовались современные возможности Kotlin для улучшения читаемости, удобства сопровождения и безопасности кода.

Подробное описание:

- Конфигурация системы сборки: Миграция началась с обновления конфигурации системы сборки - обновлен build.gradle.kts для

поддержки компиляции Kotlin и настроены плагины Kotlin в `gradle/libs.versions.toml`.

- Миграция классов: Все классы Java были систематически преобразованы в Kotlin с использованием возможностей языка Kotlin: `null safety`, `data classes`, первичные конструкторы, функции расширения и лямбда-выражения.
- Особое внимание: Особое внимание было уделено рефакторингу сущностей и слоя репозитория. Сущности унаследованы от базовых классов `AbstractEntity`, `IntIdEntity`, `EmbeddedIdEntity`. Поля перемещены в первичные конструкторы со значениями по умолчанию. Сохранена совместимость аннотаций JPA. Использование `Optional` заменено на использование `nullable` типов.

Принесённая ценность:

- `Null safety`: Исключение `NullPointerException` во время компиляции через систему типов Kotlin
- Краткость: Сокращение шаблонного кода с помощью `data classes`, свойств и вывода типов
- Совместимость: Бесшовная интеграция с существующими библиотеками и фреймворками Java сохранена
- Удобство сопровождения: Более читаемый и выразительный код с меньшим количеством формальностей
- Оптимизированная разработка: Современные языковые возможности ускорили рабочие процессы разработки

2.2. Внедрение типобезопасной конфигурации

[Pull request](#)

Задача

Проблема: Использование `@Value` аннотаций является bad practice: отсутствие типобезопасности, централизации и валидации конфигурации.

Цель:

- Обеспечить типобезопасность конфигурационных параметров
- Создать централизованную конфигурацию
- Улучшить валидацию конфигурации на старте приложения
- Получить поддержку автодополнения в IDE

Результаты

Заменить `@Value` аннотации на типобезопасные `@ConfigurationProperties` классы с валидацией через Bean Validation.

Что было сделано:

- Создан класс `SecurityConfigProperties` с аннотациями `@ConfigurationProperties` и `@Validated`
- Добавлены аннотации валидации (`@NotBlank`, `@Positive`) для значений конфигурации
- Обновлен `JwtService` для использования `constructor injection` вместо `@Value` аннотаций
- Включена обработка свойств конфигурации через `@EnableConfigurationProperties`

Принесённая ценность:

- Типобезопасность: Значения конфигурации строго типизированы с проверкой на этапе компиляции

- Валидация при старте: Jakarta Validation обнаруживает ошибки конфигурации при запуске приложения
- Централизация: Все настройки безопасности собраны в одном месте с четким разделением от бизнес-логики
- Автодополнение в IDE: IntelliJ IDEA предлагает автозавершение для конфигурационных свойств
- Расширяемость: Легко добавлять новые свойства по мере роста приложения
- Рефакторинг-friendly: IDE может автоматически переименовывать свойства

2.3. Внедрение структурного логирования

[Pull request](#)

Задача

Проблема: Отсутствие логирования затрудняет диагностику проблем и мониторинг системы.

Цель:

- Обеспечить наблюдаемость (observability) системы
- Создать машинно-читаемые логи для систем мониторинга
- Упростить диагностику проблем и отладку
- Логировать важные бизнес-события с контекстом

Реализация: Внедрить структурное логирование с KotlinLogging и Logstash Logback Encoder, настроив вывод в JSON-формат для машинной обработки.

Результаты

Внедрено структурное логирование с использованием KotlinLogging и Logstash Logback Encoder для обеспечения машинно-читаемых логов. Это обеспечивает наблюдаемость системы и упрощает интеграцию с системами мониторинга.

Что было сделано:

- Настроена конфигурация Logback в logback-spring.xml:
 - Console Appender для разработки
 - File Appender для человекочитаемых логов (logs/is-project.log)
 - JSON File Appender для машинночитаемых логов (logs/is-project.json)
- Добавлено логирование во все сервисные классы (12 файлов) с использованием паттерна withLoggingContext

- Переменные выводятся как отдельные JSON-поля для удобной фильтрации
- Настроены camelCase имена полей и ротация логов

Принесённая ценность:

- Наблюдаемость: Полное покрытие бизнес-логики логированием для отладки и мониторинга
- Машинная обработка: JSON-логи легко парсятся системами мониторинга (ELK Stack, Splunk, Datadog, CloudWatch)
- Фильтрация и поиск: Возможность поиска и фильтрации по отдельным полям (userId:123, level:ERROR)
- Два формата: Человекочитаемые логи для разработки и машинночитаемые для продакшена
- Контекстная информация: Все связанные данные логируются вместе как структурированные поля
- Готовность к масштабированию: При росте системы структурированные логи критически важны для анализа

2.4. Обновление зависимостей

[Pull request](#)

Задача

Проблема: Устаревшие версии Spring Boot и других библиотек содержат известные уязвимости и не поддерживают новые возможности.

Цель:

- Получить исправления безопасности
- Использовать новые возможности библиотек и фреймворков
- Улучшить производительность
- Обеспечить совместимость с современными инструментами
- Реализация: Обновить все зависимости проекта до актуальных версий (Spring Boot, Kotlin, Spring Doc OpenAPI, JWT, MapStruct, PostgreSQL driver).

Результаты

Обновлены все зависимости проекта до актуальных версий для получения исправлений безопасности, новых возможностей и улучшений производительности.

Что было сделано:

- Обновлен Spring Boot до версии 4.0.1
- Обновлен Kotlin до версии 2.3.0
- Обновлены библиотеки:
- Spring Doc OpenAPI
- JWT (JWT библиотека)
- MapStruct
- PostgreSQL driver
- KotlinLogging и Logback encoder

- Проверена совместимость всех обновлённых зависимостей
- Убедились, что приложение корректно работает после обновления

Принесённая ценность:

- Безопасность: Устранены известные уязвимости в старых версиях библиотек
- Новые возможности: Доступ к новым features Spring Boot 4.01. и Kotlin 2.3.0
- Оптимизация производительности: Новые версии содержат оптимизации и улучшения
- Долгосрочная поддержка: Актуальные версии получают обновления дольше
- Совместимость: Проект совместим с современными инструментами и библиотеками
- Снижение технического долга: Не накапливается отставание от актуальных версий

Презентация

- смотреть видео “Демонстрация_работающего_приложения”
- смотреть видео “Обзор_выполненной_работы_код,_документация”